

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.11.15	完成日期	2024.11.22

第一题

1. 实验题目

A small text searching engine using hash tables

2. 实验目的

Basic requirements:

- 1.Excerpt an essay ($1KB \leq \text{essay size} \leq 1MB$) from anywhere and save it as “text.txt” .
- 2.Write a program for the text searching (English & Chinese).
- 3.The program receives several queries. Each query contains a keyword.
- 4.You program should output all the sentences containing this keyword. (A sentence is ended up with a period “.”)
- 5.Test different queries (at least 10 queries) by yourselves.
- 6.Try to use different hashing functions and/or collision resolving methods.

3. 算法设计

1. 输入查找单词个数、语言、单词

2. 建立哈希表

- ①使用标准哈希函数对字符串求哈希值
- ②使用链式法
- ③链上的每一个结点为 pair，包含单词和包含单词的句子
- ④同时写一个 Search 函数，传入一个单词，对链进行遍历匹配
(不同单词可能哈希值相同, 发生冲突, 需要进行再一次查找),
返回含有该单词的所有句子的 vector

3. 读取文件

- ①先读取文件将文件的所有句子拼接成文本 content
- ②使用正则表达式切割文本 content，以句号为结束符；
- ③利用正则表达式，将所有句子放入 vector 容器，得到含有所有句子的 vector 容器

4. 遍历所有句子进行查找

- ①依次对要查找的每个单词进行遍历
- ②查找到时将 pair<单词，句子>插入链式哈希表

5. 输出结果

- ①调用哈希类里的 Search 函数，找到句子结果 result
- ②当 result 的 size==0 即找不到含有该单词的句子
当 size!=0 则依次输出 result 里的句子

6. 文档为英文与英文的不同处理策略

- ① 当文档为英文时，使用 string 即可进行操作
- ② 文档为中文时，需要使用宽字符对文档内容进行读取，因此相对应英文处理的操作方式，string 要变为 wstring，输入时将要搜索的单词从 gbk 格式变为 wstring，对字符串进行相应的操作时也要改成对 wstring 的操作方式，如 wifstream 等等

4.程序运行与测试

1.输入

```
查找单词个数：
5
查找的单词：
中国共产党
毛泽东思想
以人民为中心
工人
科学
```

2.主函数

创建哈希表、读取文件获得所有句子、对所有句子进行查找单词遍历、打印输出结果

```
Hashing0 H(Query.size());
vector<string> Sentence = ReadText0(filepath);
Traversal0(Sentence, Query, H);
print0(Query, H);
```

3.哈希表

- ①成员类型为<vector<list<pair<string,string>>>>
- ②构造函数
- ③求哈希值，利用标准哈希函数对字符串求哈希值
- ④插入哈希表的函数，利用单词对应的哈希值找到索引 index，在哈希表中插入
- ⑤查找函数，对应单词的哈希值索引找到哈希表中对应的链，对该链上的结点进行比对，含有该单词的句子放入 result，得到含有该单词的所有句子结果 result

```

class Hashing0//哈希表
{
public:
    Hashing0(int size)//构造函数，定义哈希表大小
    {
        this->table.resize(size);
    }
    int Hash(string word)//求单词的哈希值
    {
        return hash<string>() (word) % (table.size()); // 使用标准哈希函数对字符串求哈希值
    }
    void Insert(const string& word, const string& sentence)//插入
    {
        int index = Hash(word);
        table[index].push_back({ word,sentence });
    }
    vector<string> Search(const string& word)//查找
    {
        int index = Hash(word);
        vector<string> result;//将找到的句子全部放进result
        for (list<pair<string, string>>::iterator it=table[index].begin(); it != table[index].end(); it++)
        {
            if (it->first == word)
                result.push_back(it->second);
        }
        return result;
    }
private:
    vector<list<pair<string, string>>> table;//pair<单词, 句子> 使用list存储 再将一条条链放入vector中
};

```

4.读取文件获得所有句子函数

- ①打开文件
- ②getline 函数获得文件的每一行，拼接成文本字符串 string
使用正则表达式分割文本
- ③利用正则表达式的结果得到分割好的句子，放入到含有所有句子的结果 sentences

```

vector<string> ReadText0(const string& FilePath)//读取文件并按句号分割成句子
{
    ifstream file(FilePath);//打开文件
    if (!file.is_open()) {
        cerr << "Failed to open file: " << FilePath << endl;
        return {}; // 如果文件打开失败, 返回空的句子列表
    }

    string line;
    string content;
    while (getline(file, line))
        content = content+line;//拼接每一行 句子之间空格隔开

    // 一次性读取文件内容
    //如果没有适当的句子分隔(例如空格), 就可能导致匹配的句子没有明显的分隔

    // 使用正则表达式分割文本, 按句号、问号或感叹号作为结束符. 它会忽略开头的空格
    regex sentenceRegex("[^!.?]+[!.?])"); // 匹配句子的正则表达式 英文

    vector<string> sentences;

    // 使用正则表达式进行匹配, 提取所有符合条件的句子
    auto wordsBegin = sregex_iterator(content.begin(), content.end(), sentenceRegex);
    auto wordsEnd = sregex_iterator();

    /*
    sregex_iterator(content.begin(), content.end(), sentenceRegex):
    这是正则表达式的迭代器, 用于查找 content 中所有与 sentenceRegex 匹配的子串(即句子)。
    它返回一个迭代器, 指向匹配的最后一个结果。
    sregex_iterator(): 这是一个默认构造的迭代器, 表示匹配的结束位置。
    这个迭代器的作用是标记所有匹配的范围。
    */
    // 将匹配到的每个句子加入到结果向量
    for (sregex_iterator i = wordsBegin; i != wordsEnd; ++i)
    {
        string sen = i->str();
        while (sen[0] == ' ')
        {
            sen.erase(sen.begin());
        }

        sentences.push_back(sen); // 添加匹配的句子
    }

    /*
    i->str(): i 是一个迭代器, 它指向当前匹配的 std::smatch 对象。
    smatch 类是一个用于存储匹配结果的容器,
    它有一个 str() 成员函数, 可以返回当前匹配的字符串, 也就是当前的句子。
    */
    return sentences;
}

```

(此处输出存在漏洞, 句子判断.即结尾, 但有的句子是.”结尾的, 会把”给漏掉)

代码更新:

```

93 //regex sentenceRegex("[^!.?]+[!.?])"); // 匹配句子的正则表达式(中文)
94 regex sentenceRegex(R"([^\s!]+(?:[!.\s]|\s[!.\s])*)"); //优化 .”结尾的句子判断

```

5.遍历所有句子匹配单词函数: 遍历所有的句子, 查找该句子中是否含有要查找的单词, 如果有则插入, 没有则跳过不需要操作

```

void Traversal0(vector<string>& Sentence, vector<string>& Query, Hashing0& H)
{
    for (int i = 0; i < Sentence.size(); i++)
    {
        for (int j = 0; j < Query.size(); j++)
        {
            if (Sentence[i].find(Query[j])!=Sentence[i].npos)//没找到返回npos迭代器
            {
                H.Insert(Query[j], Sentence[i]);
            }
        }
    }
}

```

6.打印函数

- ①利用哈希表类里的成员函数 `Search` 得到含有某一个单词的所有句子的结果 `result`
- ②判断 `size` 的大小，为 0 则查不到含有该单词的句子，不为 0 则输出所有句子的结果。

```
void print0(const vector<string>& Query, Hashing0& H)
{
    for (int i = 0; i < Query.size(); i++)
    {
        vector<string> result = H.Search(Query[i]);
        if (result.size() == 0)
        {
            cout << "找不到包含 "<<Query[i]<<" 的句子" << endl;
            return;
        }
        cout << "包含 "<< Query[i] << " 的句子如下: " << endl;

        for (int j = 0; j < result.size(); j++)
        {
            cout <<result[j] << endl;
        }
        cout << endl;
    }
}
```

7. 测试结果

输入：在《共产党章程中》查找三个单词:以人民为中心、马克思主义、毛泽东思想

输出：对应输出含有该三个单词的句子

验证：测试结果正确，程序正确

Microsoft Visual Studio 调试 × + -

查找单词个数:
5

查找的单词:
中国共产党
毛泽东思想
以人民为中心
工人
科学

包含“中国共产党”的句子如下:
中国共产党是中国工人阶级的先锋队, 同时是中国人民和中华民族的先锋队, 是中国特色社会主义事业的领导核心, 代表中国先进生产力的发展要求, 代表中国先进文化的前进方向, 代表中国最广大人民的根本利益。

中国共产党以马克思列宁主义、毛泽东思想、邓小平理论、“三个代表”重要思想、科学发展观、习近平新时代中国特色社会主义思想作为自己的行动指南。

中国共产党人追求的共产主义最高理想, 只有在社会主义社会充分发展和高度发达的基础上才能实现。

以毛泽东同志为主要代表的中国共产党人, 把马克思列宁主义的基本原理同中国革命的具体实践结合起来, 创立了毛泽东思想。

毛泽东思想是马克思列宁主义在中国的运用和发展, 是被实践证明了的关于中国革命和建设的正确的理论原则和经验总结, 是中国共产党集体智慧的结晶。

在毛泽东思想指引下, 中国共产党领导全国各族人民, 经过长期的反对帝国主义、封建主义、官僚资本主义的革命斗争, 取得了新民主主义革命的胜利, 建立了人民民主专政的中华人民共和国; 新中国成立以后, 顺利地进行了社会主义改造, 完成了从新民主主义到社会主义的过渡, 确立了社会主义基本制度, 发展了社会主义的经济、政治和文化。

十一届三中全会以来, 以邓小平同志为主要代表的中国共产党人, 总结新中国成立以来正反两方面的经验, 解放思想, 实事求是, 实现全党工作中心向经济建设的转移, 实行改革开放, 开辟了社会主义事业发展的新时期, 逐步形成了建设中国特色社会主义的路线、方针、政策, 阐明了在中国建设社会主义、巩固和发展社会主义的基本问题, 创立了邓小平理论。

邓小平理论是马克思列宁主义的基本原理同当代中国实践和时代特征相结合的产物, 是毛泽东思想在新的历史条件下的继承和发展, 是马克思主义在中国发展的新阶段, 是当代中国的马克思主义, 是中国共产党集体智慧的结晶, 引导着我国社会主义现代化事业不断前进。

十三届四中全会以来, 以江泽民同志为主要代表的中国共产党人, 在建设中国特色社会主义的实践中, 加深了对什么是社会主义、怎样建设社会主义和建设什么样的党、怎样建设党的认识, 积累了治党治国新的宝贵经验, 形成了“三个代表”重要思想。

“三个代表”重要思想是对马克思列宁主义、毛泽东思想、邓小平理论的继承和发展, 反映了当代世界和中国的发展变化对党和国家工作的新要求, 是加强和改进党的建设、推进我国社会主义自我完善和发展的强大理论武器, 是中国共产党集体智慧的结晶, 是党必须长期坚持的指导思想。

十六大以来, 以胡锦涛同志为主要代表的中国共产党人, 坚持以邓小平理论和“三个代表”重要思想为指导, 根据新的发展要求, 深刻认识和回答了新形势下实现什么样的发展、怎样发展等重大问题, 形成了以人为本、全面协调可持续发展的科学发展观。

科学发展观是同马克思列宁主义、毛泽东思想、邓小平理论、“三个代表”重要思想既一脉相承又与时俱进的科学理论, 是马克思主义关于发展的世界观和方法论的集中体现, 是马克思主义中国化重大成果, 是中国共产党集体智慧的结晶, 是发展中国特色社会主义必须长期坚持的指导思想。

十八大以来, 以习近平同志为主要代表的中国共产党人, 坚持把马克思主义基本原理同中国具体实际相结合、同中华优秀传统文化相结合, 科学回答了新时代坚持和发展什么样的中国特色社会主义、怎样坚持和发展中国特色社会主义等重大时代课题, 创立了习近平新时代中国特色社会主义思想。

在习近平新时代中国特色社会主义思想指导下, 中国共产党领导全国各族人民, 统揽伟大斗争、伟大工程、伟大事业、伟大梦想, 推动中国特色社会主义进入了新时代, 实现第一个百年奋斗目标, 开启了实现第二个百年奋斗目标新征程。

中国共产党自成立以来, 始终把为中国人民谋幸福、为中华民族谋复兴作为自己的初心使命, 历经百年奋斗, 从根本上改变了中国人民的前途命运, 开辟了实现中华民族伟大复兴的正确道路, 展示了马克思主义的强大生命力, 深刻影响了世界历史进程, 锻造了走在时代前列的中国共产党。

中国共产党在社会主义初级阶段的基本路线是: 领导和团结全国各族人民, 以经济建设为中心, 坚持四项基本原则, 坚持改革开放, 自力更生, 艰苦创业, 为把我国建设成为富强民主文明和谐美丽的社会主义现代化强国而奋斗。

中国共产党在领导社会主义事业中, 必须坚持以经济建设为中心, 其他各项工作都服从和服务于这个中心。

坚持社会主义道路、坚持人民民主专政、坚持中国共产党的领导、坚持马克思列宁主义毛泽东思想这四项基本原则, 是我们的立国之本。

中国共产党领导人民发展社会主义市场经济。

中国共产党领导人民发展社会主义民主政治。

坚持和完善人民代表大会制度、中国共产党领导的多党合作和政治协商制度、民族区域自治制度以及基层群众自治制度。

中国共产党领导人民发展社会主义先进文化。

中国共产党领导人民构建社会主义和谐社会。

中国共产党领导人民建设社会主义生态文明。

中国共产党坚持对人民解放军和其他人民武装力量的绝对领导, 贯彻习近平强军思想, 加强人民解放军的建设, 坚持政治建军、改革强军、科技强军、人才强军、依法治军, 建设一支听党指挥、能打胜仗、作风优良的人民军队, 把人民军队建设成为世界一流军队, 切实保证人民解放军有效履行新时代军队使命任务, 充分发挥人民解放军在巩固国防、保卫祖国和参加社会主义现代化建设中的作用。

中国共产党坚持和发展平等团结互助和谐的社会主义民族关系, 积极培养、选拔少数民族干部, 帮助少数民族和民族地区发展经济、文化和社会事业, 铸牢中华民族共同体意识, 实现各民族共同团结奋斗、共同繁荣发展。

中国共产党同全国各民族工人、农民、知识分子团结在一起, 同各民主党派、无党派人士、各民族的爱国力量团结在一起, 进一步发展和壮大由全体社会主义劳动者、社会主义事业的建设者、拥护社会主义的爱国者、拥护祖国统一和致力于中华民族伟大复兴的爱国者组成的最广泛的爱国统一战线。

中国共产党坚持独立自主的和平外交政策, 坚持和平发展道路, 坚持互利共赢的开放战略, 统筹国内国际两个大局, 积极发展对外关系, 努力为我国的改革开放和现代化建设争取有利的国际环境。

输入: 在关于 AI 的文章中查找一个单词 AI

输出: 对应含有 AI 的句子

验证: 测试结果正确, 程序正确

Microsoft Visual Studio 调试 × + -

查找单词个数:
1

查找的单词:
AI

包含“AI”的句子如下:
As we stand at the cusp of a new era, the rapid advancements in Artificial Intelligence (AI) are transforming industries, economies, and societies in ways that were once confined to science fiction.

The increasing prevalence of AI technology has sparked debates and discussions across various domains – from ethics to economics, from healthcare to education.

But amidst the excitement of new possibilities, there are deep, pressing questions about how AI will reshape our world.

How will AI affect employment, privacy, and our understanding of creativity?

At its core, AI is about building systems that can learn, adapt, and make decisions with minimal human intervention.

AI systems are already performing tasks ranging from simple data processing to complex decision-making, and their capabilities are growing exponentially.

Autonomous vehicles, AI-powered medical diagnostics, and algorithm-driven financial systems are just a few examples of how AI is already integrated into the fabric of modern life.

The impact of AI is not only economic but also philosophical.

As AI becomes more autonomous, it raises fundamental questions about what it means to be human.

Will the widespread adoption of AI lead to a more equitable society, or will it exacerbate existing inequalities?

These are questions that policymakers, technologists, and ethicists must grapple with in order to ensure that AI's benefits are distributed fairly across all segments of society.

As AI continues to evolve, one of the most pressing concerns is its ethical implications.

With AI making crucial decisions in areas like healthcare, criminal justice, and finance, the question arises: who is responsible when these systems make mistakes?

Is it the developers who built the algorithms, the companies that deploy them, or the AI itself?

In the field of healthcare, for example, AI-driven systems are increasingly being used to assist in diagnosing diseases, recommending treatments, and even managing patient care.

What if an AI system makes a mistake in diagnosing a patient, leading to a wrongful treatment plan?

Moreover, if an AI system was trained on biased data, could it perpetuate or even amplify existing inequalities in healthcare?

If an AI model is trained on historical criminal data that reflects systemic biases, it could inadvertently perpetuate racial or socioeconomic inequalities.

One of the most contentious ethical issues surrounding AI is the concept of “explainability.”

Many AI systems, particularly deep learning models, operate as “black boxes,” making decisions without offering clear explanations for how they arrived at a particular conclusion.

Beyond ethical considerations, AI is poised to have a profound impact on the global economy.

Automation, powered by AI, is already transforming industries by performing tasks more efficiently and at a lower cost than humans.

In manufacturing, for instance, AI-driven robots are revolutionizing production lines, reducing the need for human labor in repetitive and physically demanding tasks.

Similarly, in agriculture, AI-based systems are being used to optimize crop yields, monitor soil health, and even detect diseases in plants, leading to more efficient farming practices.

As AI systems take over routine tasks, there is growing fear that large numbers of jobs will be lost, particularly in industries like manufacturing, retail, and transportation.

While new jobs may be created in AI-related fields, many workers may lack the skills needed to transition into these roles, leading to increased unemployment and social inequality.

The rise of AI could also exacerbate the wealth gap between large corporations and small businesses.

Major tech companies that develop and deploy AI technologies stand to gain significantly from these innovations, while smaller firms may struggle to keep up.

Moreover, AI has the potential to reshape global trade and employment patterns.

As countries and companies race to develop and deploy cutting-edge AI technologies, those with the most advanced capabilities will likely gain a competitive advantage in the global marketplace.

This could lead to new power dynamics in international relations, where nations that excel in AI research and development could dominate industries and shape global economic policies.

While many discussions about AI focus on its potential to replace human labor, there is also growing interest in the role of AI in enhancing human creativity.

In fields like art, music, and literature, AI systems are being used to generate new works of creativity, often blending human input with machine-generated outputs.

For instance, AI algorithms have been trained to compose music, write poetry, and even create visual art.

In some cases, these AI-generated works have been showcased in galleries, sold at auction, or even performed in concert halls.

Can AI ever truly be creative, or is it simply mimicking patterns learned from human creativity?

Some argue that AI's role in creativity could be seen as a tool to augment and enhance human imagination rather than replace it.

For example, AI can assist artists in generating ideas, exploring new creative possibilities, or even overcoming creative blocks.

Similarly, in industries like film, advertising, and video game design, AI can be used to automate certain aspects of the creative process, allowing human creators to focus on higher-level decision-making and conceptual work.

However, others caution against the overreliance on AI in creative fields.

There is concern that the widespread use of AI-generated content could dilute the authenticity and emotional depth of human-created art.

中文和英文文件的输出结果分别保存在测试结果文本中。

5.在项目过程中生疏点

1.对于中文文档的操作之输入内容的 GBK 型转为宽字符型

```
wstring GbkToWstring(const string& gbkStr) {  
    int len = MultiByteToWideChar(CP_ACP, 0, gbkStr.c_str(), -1, nullptr, 0);  
    wstring wstr(len - 1, L'\0');  
    MultiByteToWideChar(CP_ACP, 0, gbkStr.c_str(), -1, &wstr[0], len);  
    return wstr;  
}
```

2.对于中文文档的操作之 string 类型改为 wstring 类型,英文为窄字符,中文为宽字符,避免操作过程中的中文乱码出现

3.标准库里的哈希函数使用

```
int Hash(string word)//求单词的哈希值  
{  
    return hash<string>()(word) % (table.size()); // 使用标准哈希函数对字符串求哈希值  
}
```

4.文件的读写: ifstream 打开文件, close 关闭文件

```
ifstream file(filePath);//打开文件  
if (!file.is_open()) {  
    cerr << "Failed to open file: " << filePath << endl;  
    return {};  
} // 如果文件打开失败, 返回空的句子列表  
  
file.close();
```

4.getline 函数获得文档中的每一行,再利用 string 的+即可获得文本的字符串

```
string line;  
string content;  
while (getline(file, line))  
    content = content+line;//拼接每一行 句子之间空格隔开
```

5.对于中文文档的操作之支持中文文档环境和宽字符打开文件


```

locale::global(locale("")); // 保证加载文件支持中文路径
wifstream file(filePath); // 打开文件
if (!file.is_open()) {
    cerr << "Failed to open file: " << filePath << endl;
    return {}; // 如果文件打开失败，返回空的句子列表
}

```

6. 高效读取文档内容操作

```

wstring content((istreambuf_iterator<wchar_t>(file)), istreambuf_iterator<wchar_t>());

```

7. 正则表达式

```

// 使用正则表达式分割文本，按句号、问号或感叹号作为结束符. 它会忽略开头的空格
regex sentenceRegex("[.!?]+[.!?]"); // 匹配句子的正则表达式 英文

vector<string> sentences;

// 使用正则表达式进行匹配，提取所有符合条件的句子
auto wordsBegin = sregex_iterator(content.begin(), content.end(), sentenceRegex);
auto wordsEnd = sregex_iterator();
/*
sregex_iterator(content.begin(), content.end(), sentenceRegex):
    这是正则表达式的迭代器，用于查找 content 中所有与 sentenceRegex 匹配的子串（即句子）。
    它返回一个迭代器，指向匹配的第一个结果。
sregex_iterator(): 这是一个默认构造的迭代器，表示匹配的结束位置。
    这个迭代器的作用是标记所有匹配的范围。
*/
// 将匹配到的每个句子加入到结果向量
for (sregex_iterator i = wordsBegin; i != wordsEnd; ++i)
{
    string sen = i->str();
    while (sen[0] == ' ')
    {
        sen.erase(sen.begin());
    }
    sentences.push_back(sen); // 添加匹配的句子
}
/*
i->str(): i 是一个迭代器，它指向当前匹配的 std::smatch 对象。
    smatch 类是一个用于存储匹配结果的容器，
    它有一个 str() 成员函数，可以返回当前匹配的字符串，也就是当前的句子。
*/

```

6.实验总结与心得

1.优点：①链式哈希表可以有效解决冲突的问题，实现比较简单，且可以动态扩展，对于未知元素数量较友好②查找准确，可以正确地得到需要的文本结果。

2.缺点：①链式哈希表的内容存储开销较大，且查找的性能较差，数据较多时影响程序性能②文档的中文和英文需要对应两套不同的操作，尝试将两种处理方式合二为一，但总是因为字符识别失败、中文乱码等无法实现。

3.心得：对于这种整体的项目，还是无法独立全部完成，需要借助网络工具构思模块以及使用一些解决相应问题的陌生的库函数，但是经过这样的学习也能有不少收获，能让自己锻炼思维能力，在解题中学习知识，逐渐进步。

4.总结：实验难度对于算法上的考验难度不是很大，重点是对于几个模块函数内操作部分的熟悉，时间花费较多的是在中文文档的处理上，在学习了一些库函数的使用之后也顺利解决了。

7.提升和优化：使用不同的哈希函数的时间效率

1.测试时间的方法

```
clock_t start, end;
double cpu_time_used;

start = clock(); // 开始计时

// 你的算法代码放在这里

end = clock(); // 结束计时

cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC; // 计算时间
printf("程序执行时间为: %f seconds\n", cpu_time_used);
```

测试样例全部使用在关于 AI 的文章中搜索 artificial、AI、people 这三个单词

2.直接定址法和字符串数值哈希法和除留余数法的比较（拉链法的哈希冲突解决方法）

①直接定地址法(在原来的哈希表里的求哈希值函数做替换即可)

```
/*
int Hash(string word)//求单词的哈希值:标准哈希函数
{
    return hash<string>()(word) % (table.size());
}
*/

int Hash(string word) //求单词的哈希值:使用除留余数法
{
    int hashValue = 0;
    for (char c : word) {
        hashValue += c; //累加字符的ASCII值
    }
    return hashValue % (table.size());
}
```

时间 14710

②字符串数值哈希法

```
int Hash(string word) //求单词的哈希值:字符串数值法
{
    int hashValue = 0;
    int prime = 31; //用质数作为权重因子
    for (int i = 0; i < word.length(); i++) {
        hashValue = hashValue * prime + word[i]; //累加字符的数值权重
    }
    return abs(hashValue) % (table.size());
}
```

时间 8169

③除留余数法

```
int Hash(string word) //求单词的哈希值: 除留余数法
{
    unsigned long hashValue = 0;
    int prime = 37;
    for (int i = 0; i < word.length(); i++) {
        hashValue = (hashValue * prime + word[i]) % (table.size());
    }
    return hashValue % (table.size());
}
```

时间 16150

3.时间效率的比较（多次进行大小容量数据测试）

①直接定址法

这种方法通过简单地将字符串中每个字符的 ASCII 值相加来计算哈希值，因此查找和插入操作都非常高效，时间复杂度为常数 $O(1)$ 。在字符集较小的情况下，直接定址法表现出极高的时间效率，适合于快速查找和插入。

②字符串数值哈希法

通过将每个字符的数值加权求和，字符串数值哈希法可以有效处理较复杂的字符集，并且在字符串较短时能够提供相对均匀的哈希分布，减少冲突问题。相较于直接定址法，它提供更灵活的哈希值计算。

③除留余数法

除留余数法是一种广泛应用的哈希方法，能够通过合适选择质数和表大小来降低碰撞率，并且具有较好的扩展性，适合处理大规模数据集。通过除留余数的方式计算哈希值，通常能够保证哈希值均匀分布。

4.关于三种哈希函数的测试输出结果保存在输出文档中，只取了上面三个输入作为例子，其他大量测试已自行完成，给出上面对于三种哈希函数的总结性的结果

附录、提交文件清单

1. 测试文本: text0 《圣经》、text1 《中国共产党章程》
- 2.测试结果: test0、test1
- 3.代码: test.hpp、test.cpp
- 4.实验报告